Tutorial 2

1. Modify the following code to improve its cohesion.

```
public class Staff {
    public Speaker saveSpeaker(Speaker speaker);

    public boolean removeSpeaker(int speakerId);

    public Speaker findById(int speakerId);

    public void updateSpeakerPhoto(int speakerId, String photoUrl);

    public Product saveProduct(Product product);

    public void deleteProducts(List<Integer> productsId);

    public boolean deleteProduct(int productId);
}
```

2. Modify the following code to lower its coupling.

```
public class ParttimeStaff {
    public double getWorkedHours(){
        ....
    }

}

public class Payroll {
    private ParttimeStaff staff;

    public Payroll(ParttimeStaff ps){
        this.staff = ps;
    }

    public double CalculatePay(){
        return staff.getWorkedHours() * .......
    }
}
```

```
public class Car  {
    public void move() {
         System.out.println("Car is moving");
    }
}

class Traveler {
    Car c = new Car();
    public void startJourney() {
         c.move();
    }
}
```

3. What are the problem with a program with long methods (multiple responsibilities)?

Answer:

- Hard to reuse
- Lead to duplication
- Difficult to understand
- Making change to the code becomes riskier
- Often clutter with codes
- Hard to test. Too many possibilities
- Difficult to navigate and hard to find a specific code
- Involve in the modification frequently
- Difficult to debug

4. A software system is considered to have a "good design" if the cost of changing it is minimized. Discuss how the concepts of Cohesion and Coupling contribute to achieving this goal.

   Answer
   Cohesion (Focus): High cohesion means a module or class has a single, well-defined purpose. When requirements change, a highly cohesive system is easier to maintain because the changes are localized to specific, predictable places. If a class is "low cohesion" (doing too many unrelated things), changing one feature might accidentally break an unrelated one.
   Coupling (Dependency): Coupling refers to how much one class knows about or relies on another.
   Tight Coupling: If Class A depends on the internal details of Class B, any change to B forces a change in A.
   Loose Coupling: By using interfaces instead of concrete implementations, we decouple the classes. Class A only knows "what" Class B can do, not "how" it does it. This minimizes the "ripple effect" where a single change necessitates a cascade of edits across the codebase.
   As a conclusion, a design with High Cohesion and Low Coupling ensures that the system is modular, making it cheaper and safer to evolve over time.

5. A developer is designing a "Machine" hierarchy where different machines can have various combinations of movement (Fly, Swim) and combat capabilities (Laser, Rifle). Explain the "combinatorial explosion" problem that arises when using pure inheritance for this scenario and describe how using Composition can solve it.

   Answer

   When using pure inheritance to model machines with multiple independent traits (Movement: Fly/Swim and Weapon: Laser/Rifle), the number of required classes grows exponentially. You are forced to create a class for every possible combination: FlyLaserMachine, FlyRifleMachine, SwimLaserMachine,

and SwimRifleMachine. If you add just one more movement type (e.g., Crawl), you must manually create even more subclasses. This results in a bloated, rigid hierarchy where code is duplicated across branches.

As a solution, instead of an "is-a" relationship, we use a "has-a" relationship.

- Define interfaces for behaviors: IMoveBehavior and IWeaponBehavior.
- The Machine class contains (is composed of) references to these interfaces rather than inheriting from them.
- At runtime, you can "plug in" different implementations (e.g., a FlyBehavior and a LaserBehavior) into a generic Machineobject.

The benefit of use “has-a” relationship: This solves the explosion because you only need one Machine class and a few behavior classes. You can create any combination of machine dynamically without writing new class definitions, leading to much lower coupling and higher flexibility.

6. According to SOLID principles, which is the most suitable situation to use inheritance?

   Answer
   If an object of B should be used anywhere an object of A is used, then use inheritance.

7. According to SOLID principles, which is the most suitable situation to use composition?

   Answer
   If an object of B should use an object of A, then use composition.

8. Explain in detail the problem when the return type in an override method of a subclass is a long data type while the return type in the method of the superclass is an int data type?

   Answer
   There exist an unexpected error in the client software when the subclass is replacing its superclass. In its original design, the client is expecting smaller range of value. However, after the class is replaced by its subclass, the subclass return larger range of value which the client may not be able to accommodate.

9. Discuss the Single Responsibility Principle (SRP) and provide an example of how it can be applied in a software design.

   Answer
   The Single Responsibility Principle (SRP) states that a class should have only one reason to change. This means that a class should have only one responsibility, i.e., only one potential source of change. For example, a class that handles user authentication should not also be

responsible for sending email notifications. These responsibilities should be separated into distinct classes to adhere to SRP.

10. How does the Open/Closed Principle (OCP) contribute to software maintainability and extensibility? Provide a real-world example to illustrate this principle.
    Answer
    The Open/Closed Principle (OCP) states that software entities should be open for extension but closed for modification. By adhering to OCP, developers can create software components that can be extended without modifying their source code, which contributes to better maintainability and extensibility. For example, using abstract classes or interfaces to define general behavior that can be extended by concrete implementations without modifying the existing code showcases OCP in action.

11. Detail the Liskov Substitution Principle (LSP) and explain its significance in the context of inheritance and polymorphism.
    Answer
    The Liskov Substitution Principle (LSP) states that objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program. This principle is significant in the context of inheritance and polymorphism as it ensures that subtypes should be substitutable for their base types without altering the desirable properties of the program.

12. Explain the Interface Segregation Principle (ISP) and discuss its role in software design.
    Answer
    The Interface Segregation Principle (ISP) states that a client should not be forced to implement an interface that it doesn't use. By creating specific interfaces for specific client requirements, ISPs promote modularity and reusability by allowing for smaller, more focused interfaces that cater to the specific needs of each client class.

13. Describe how the Dependency Inversion Principle (DIP) helps to decouple software modules and enhance flexibility in a software architecture.
    Answer
    The Dependency Inversion Principle (DIP) emphasizes that high-level modules should not depend on low-level modules, but both should depend on abstractions. DIP helps to decouple software modules and enhance flexibility by relying on abstractions rather than concrete implementations. For example, depending on an abstract database interface rather than a specific database implementation allows for different database implementations to be used without modifying the high-level module.

14. What is the problem with the following interface? Improve

```
public interface FileStorage {
    public String storeFile(MultipartFile file, FileType fileType, Integer id);
    public Resource loadFileAsResource(String fileName, FileType fileType);
    public boolean removeFile(String fileName, FileType fileType);
    public DecodedJWT decodeJWT(String jwtString) throws Exception;
    public String GenerateJWT(UserDetails user);
    public String GenerateRefreshJWT(UserDetails user);

}
```

it.